

DRASTHA

Understand it.

Explain it.

Defend it.

A plain-English guide to passive threat detection, technical architecture, machine learning, evidence, APIs and storage.

Technical presentation and evaluator preparation handbook

Prepared from the local source repository on 4 September 2026. Baseline commit: 3c75635. This is a guide to what the prototype actually implements, not a promise that every planned production capability exists.

The most important idea: Drastha is a passive, metadata-based, hybrid threat-analysis prototype. It combines explainable behavioural rules, statistical measurements, and a small supervised DGA classifier. It produces intelligence for an analyst, not automatic blocking commands.

Study route: first read chapters 1-5 for foundations and architecture; then 6-14 for detectors and ML; then 15-22 for scoring, APIs, storage and the dashboard. Use chapters 23-28 for evaluation, limitations and presentation practice.

Three statements to remember:

- Healthy telemetry does not mean harmless traffic. A perfectly valid replay can contain attacks.
- Confidence is not the same as severity, risk score or measured accuracy.
- Eight correct behaviours in a controlled fixture do not prove 100% real-world detection accuracy.

This handbook includes real upload-mode thresholds, an exact risk-score example, training sample counts, a trace from browser upload to evidence drawer, and honest answers to difficult evaluator questions.

YOUR LEARNING MAP

Contents

1. The problem and the solution in plain English 3
2. Networking knowledge you need first 4
3. Which attacks are handled? 5
4. The complete technical architecture 6
5. From a selected file to canonical events 7
6. Validation, chronology and data quality 8
7. Sliding windows, thresholds and cooldowns 9
8. Reconnaissance and port scanning 10
9. SYN flooding and distributed-source flooding 11
10. UDP flooding and amplification 12
11. Botnet C2 beaconing 13
12. DGA detection: ML and lexical fallback 14
13. DNS tunnelling 15
14. Encrypted-session anomaly and exfiltration 16
15. The ML model: exactly how it is trained 17
16. Training validation and honest accuracy claims 18
17. How false alerts are reduced 19
18. From detector output to a standardized alert 20
19. Incident correlation, severity and risk scoring 21
20. APIs: the bridge between browser and backend 22
21. Database design and configuration 23
22. How the result reaches the dashboard 24
23. Dashboard glossary: what each term really means 25
24. How to read the full evidence page 26
25. Evaluation: records are not behaviours 27
26. Throughput, tests and reproducibility 28
27. Difficult evaluator questions and safe answers 29
28. Presentation script and final revision sheet 30
29. Source map for technical follow-up 31

Priority reading: chapters 4-19 explain the architecture, detector rules, ML and scoring. Chapters 23-28 prepare you to narrate the dashboard and answer questions.

FOUNDATIONS & ARCHITECTURE

1. The problem and the solution in plain English

Critical infrastructure operators need to observe traffic without creating a new route into their protected network. A monitoring system that can connect back to production could become an attacker pivot if compromised. The desired architecture sends a copy of observed traffic into a separate monitoring enclave, using passive mirroring or a hardware data diode.

A data diode enforces one-directional transfer at the hardware boundary. A mirror copies traffic but does not, by itself, prove there is no return path. Drastha does not implement a physical diode; it is the analytics software intended to operate behind that boundary. Deployment must enforce isolation separately.

Drastha reads observed records, validates them, extracts features, runs threat-specific detectors, attaches confidence and evidence, groups related alerts, assigns investigation priority, saves intelligence, and shows it on a dashboard.

The system never needs to ping the observed host, open a TCP connection to it, resolve an observed domain, complete its handshake, decrypt TLS, or issue a block command. Browser-to-API and API-to-database communication happens inside the monitoring/application side. That does not represent a return path to the traffic source.

What the prototype supports today:

- JSON/JSONL/NDJSON replay upload and a paced simulated metadata stream.
- Local PCAP conversion through an installed Zeek executable, including a WSL runner.
- Connection, visible DNS and supplied TLS/QUIC metadata analysis.
- Explainable alerts, incident correlation, a local database, feedback and evidence export.

What it does not prove: an installed physical one-way network boundary, line-rate capture performance, production-grade authentication, comprehensive malware identification, or perfect accuracy on unknown traffic.

Evaluator answer: "We move security analysis into an isolated monitoring side. We use only passive observations, so even the detection logic does not depend on contacting the monitored systems. The result is labelled, scored intelligence for an analyst."

FOUNDATIONS & ARCHITECTURE

2. Networking knowledge you need first

An IP address identifies a network endpoint. IPv4 examples look like 192.0.2.10; IPv6 examples look like 2001:db8::10. An address is not always one physical device: NAT, proxies and shared infrastructure can hide the actual origin.

A port identifies a transport service endpoint. For example, 443 commonly carries HTTPS and 53 commonly carries DNS. A port is a clue, not proof of application identity. Applications can use unusual ports.

TCP normally establishes a connection using SYN, SYN-ACK and ACK. A passive sensor watches these exchanges; it does not complete them. Many incomplete attempts may suggest scanning or flooding, but can also mean an outage, rejection, packet loss or incomplete visibility.

UDP has no TCP-style connection handshake. DNS and other services often use it. An attacker can exploit services that return a response much larger than the request; this is amplification. Reflection means the response is directed toward a victim, often through a forged source address.

A packet is one unit transmitted on a network. A flow record summarizes an exchange: endpoints, protocol, time, bytes, packets and state. One flow can represent many packets. Therefore 1,000 flows/second is not the same as 1,000 packets/second or 1,000 Mbps.

DNS translates names into information such as IP addresses. A DNS query has a name and record type, for example A, AAAA or TXT. TLS protects application content. QUIC commonly runs over UDP and uses TLS-based security. Visible handshake metadata and externally derived timing/size features can still support passive analysis without content decryption.

Zeek is a network analysis tool. It can read a capture and produce structured connection, DNS and encrypted-session logs. Drastha consumes the resulting metadata; Zeek and Drastha have different jobs.

Important direction rule: Zeek `orig_bytes` means bytes from the connection originator, and `resp_bytes` means bytes from the responder. In the code these become outbound and inbound bytes. They are not automatically "leaving our organization" and "entering our organization" unless deployment context establishes that orientation.

FOUNDATIONS & ARCHITECTURE

3. Which attacks are handled?

The specification contains six broad threat areas, but the mixed evaluation separates DDoS subtypes and DNS behaviours into eight intended attack behaviours.

Behaviour	What it means	Main passive evidence
SYN / distributed-source flood	Many incomplete TCP attempts pressure a service	Count, incomplete ratio, port concentration, source diversity
UDP reflection/amplification	Responses greatly exceed requests	UDP flow count and responder/originator byte ratio
Reconnaissance	A source searches hosts or ports	Unique destination ports and hosts in a short window
C2 beaconing	A potentially compromised host repeatedly calls a controller	Regular timing, similar small exchanges, repetition
DGA domains	Software generates many possible rendezvous names	Character n-grams and repeated suspicious lexical patterns
DNS tunnelling	DNS names or responses carry nonstandard encoded communication	Query count, subdomain variety, length, entropy and TXT share
Encrypted-session anomaly	Encrypted traffic metadata is suspicious	Repeated rare fingerprint plus supplied size/timing anomalies
Data exfiltration	A host may be transferring information outward	Large originator volume, asymmetry and source baseline

Classification uses three related fields. `threat_type` is the broad family, such as `denial_of_service`. `subtype` is a more specific machine identifier, such as `distributed_source_syn_flood`. `threat_class` is the display/interoperability label; some subtypes have an explicit human-readable mapping and other cases fall back to the broad family.

The detectors are not mutually exclusive. A source can scan, contact a C2 server, and later transfer data. Different detectors can legitimately contribute to one incident. The upload path also resolves a specific overlap where scan-like flows would otherwise be labelled as a SYN flood.

Do not say "we prove spoofing". High source diversity supports a distributed-source description. The passive records do not prove that the source addresses are forged.

Do not say "we decrypt and find malware". The actual label is encrypted-session metadata anomaly. It is evidence consistent with suspicious behaviour, not definitive malware attribution.

FOUNDATIONS & ARCHITECTURE

4. The complete technical architecture

There are three entry paths. They share important normalizers and detectors, but they are not identical end-to-end implementations.

UPLOAD PATH: browser file selection -> JSON request to FastAPI -> container parser -> canonical alias normalization -> schema routing -> validation/quality -> normalized event lists -> detectors -> deduplication -> optional evaluation -> incident correlation -> repository -> JSON response -> React dashboard.

SIMULATED STREAM: known local fixtures -> normalized and sorted events -> paced event loop -> detector processing per event -> incident update and database write -> Server-Sent Events -> live dashboard. The simulator preloads its fixture list; it is a demonstration of incremental inference and publication, not a production unbounded ingestion service.

PCAP PATH: local capture file -> Zeek subprocess using file-read mode -> generated JSON logs -> CLI normalizers and selected detectors. The current PCAP CLI feeds the generated conn.log into connection detection; do not claim it automatically fuses every generated DNS/TLS log into the upload route.

The main internal objects are NetworkEvent, DNSEvent, EncryptedSessionMetadata, Alert and Incident. They are frozen Python dataclasses. They make field names and types consistent, although nested raw dictionaries are not deep immutable.

The detector interface is essentially process(event) -> zero or more alerts. Each detector owns its observation window and cooldown state. A window is keyed by the entity relevant to that threat: source for recon, destination for DDoS, source/destination/service for C2, or source/base-domain for tunnelling.

Technology choices:

- Python standard library for the analytics core: collections, math, statistics, dataclasses, json and hashlib.
- FastAPI, Pydantic and Uvicorn for HTTP request validation and serving.
- SQLite by default; an optional psycopg PostgreSQL repository adapter.
- React and TypeScript for the browser interface, Vite for frontend builds, Lucide for icons.
- unittest for automated verification. No TensorFlow, PyTorch or scikit-learn is required by the current custom DGA classifier.

The software is a modular prototype, not a distributed Kafka/Spark architecture. Those components should not be presented as implemented.

FOUNDATIONS & ARCHITECTURE

5. From a selected file to canonical events

The browser reads the selected file as text. It sends POST /api/replays/analyse with a JSON body containing filename and content. This outer API request is a transport wrapper; the text inside content can itself be JSONL, a JSON array, a wrapped records array, or one object. The browser does not convert a manifest into traffic.

Example transport concept: filename is "traffic.jsonl" and content is the text of that file. FastAPI's ReplayUploadRequest checks basic body shape and size. The upload function additionally checks the extension, a 5,000,000-byte UTF-8 limit and a 20,000-record limit.

The parser extracts records from these containers:

- JSONL/NDJSON: one JSON object per nonblank line.
- JSON array: a list of record objects.
- Wrapped JSON: an object with records containing a list.
- Single object: one record.

A manifest such as {"dataset":"traffic.jsonl","records":452} describes a file; it does not contain its observations. It is rejected with an explanatory message.

Canonical connection fields are ts, uid, id.orig_h, id.resp_h and proto. Source/destination ports are optional. Timestamp aliases include timestamp, time and @timestamp. Source aliases include src_ip, src, source_ip and source.address; destination aliases have corresponding dst forms. flow_id/flowid map to uid, and protocol/network.transport map to proto. Literal dotted and supported nested paths are recognized.

The normalizer creates a copy rather than editing the original file. It preserves other fields, including service, duration, byte/packet counters and metadata. Conflicting aliases are rejected rather than guessed. The current alias comparison is conservative textual comparison; differently formatted equivalent timestamps can still be treated as conflicting.

Schema routing is not threat classification. A visible query/query_name routes to DNS. Fingerprint fields route to TLS or QUIC, with explicit transport used to distinguish QUIC. Otherwise it is treated as a connection candidate. Upload DNS records are not also processed as generic connections. TLS records currently pass through connection normalization too, so upload TLS records need connection-required fields; the dedicated encrypted-log reader is a separate path.

FOUNDATIONS & ARCHITECTURE

6. Validation, chronology and data quality

Validation asks "Can we trust the shape and sequence of these observations?" Detection asks "Does the observed behaviour resemble a threat?" They answer different questions.

The current normalizers check required fields, IPv4/IPv6 addresses, port range and supported protocol names. Timestamps accept numeric Unix seconds or ISO-8601; timezone-aware values are converted to epoch seconds, and timezone-naive ISO values are treated as UTC. Missing optional ports default to zero.

When a recoverable record-level failure occurs, the upload loop increments rejected and quarantined counters and records a structured reason with line/record location, category, field and value where available. A short error sample is shown in the dashboard. Quarantine here means rejection accounting in the response, not a durable quarantine archive.

Quality policy:

- **Healthy:** accepted data without the tracked rejection, ordering or duplicate issues.
- **Degraded:** up to 10% rejected records, or out-of-order observations, or duplicate flow IDs.
- **Unusable:** no accepted events or more than 10% rejected records. Detection is not run for an unusable upload.

Some malformed whole JSON documents fail before recoverable per-line processing. Do not promise that every possible corrupt file is partially recovered. Unusable uploads return HTTP 422 with a reason, not a full persisted quality report.

Original order is checked before detector sorting. The monitor compares each accepted timestamp with the largest timestamp already seen. Thus one backwards jump can make several later records count as out of order. Equal timestamps are allowed. Sorting events for inference does not erase the original-quality flag.

Your original fixture placed BMON0007 at 08:17:00 before BUPL0001 at 08:11:40: a 320-second backwards step. There was one adjacent backwards transition but ten records behind the maximum seen timestamp. The chronological v3 keeps all 452 records and has zero such records.

Exact duplicate UID/content and duplicate UID/different content are reported separately. They are not automatically removed from detector input. This preserves observations but means duplication can still influence counts; ingestion deduplication needs explicit deployment semantics.

The checks are useful prototype checks, not an exhaustive hostile-input security audit. Resource limits, finite numeric checks, strict integer semantics and durable audit trails need additional production hardening.

FOUNDATIONS & ARCHITECTURE

7. Sliding windows, thresholds and cooldowns

A sliding window means "look at recent observations for this entity." A 10-second recon window at event time 100 considers contacts approximately from time 90 onward. When newer observations arrive, older items fall outside the window.

Event time is the timestamp inside the traffic record. Processing time is when Python handles it. A replay can cover hours of event time while processing in milliseconds. A six-connection C2 pattern requiring a 30-second observation span may therefore alert quickly during accelerated replay, but must wait for enough real event-time evidence during live monitoring.

The window implementation uses dictionaries and deques. Keys separate relevant entities. For example, unrelated source/destination C2 conversations should not be mixed into one timing series. Counts and statistics are recomputed from each current window. This is straightforward and inspectable but is not constant-cost processing at arbitrarily high rates.

A threshold is a decision boundary. It is not inherently a learned value. Most current thresholds are manually configured prototype settings. A cooldown is a period during which an already-detected entity/subtype does not emit another near-identical alert. It reduces repeated notifications, not the original false-positive probability.

Detector	Main window	Key	Cooldown
Recon	10 seconds	Source IP	30 seconds per source/subtype
DDoS	5 seconds	Destination IP	30 seconds per destination/subtype
C2	300 seconds	Source, destination, port, protocol	300 seconds
DNS	60 seconds	Source/base-domain; source for lexical DGA	120 seconds
Encrypted anomaly	60 seconds	Source, destination, fingerprint	120 seconds
Exfiltration	300 seconds	Source/destination; source baseline	600 seconds

Upload demonstration overrides matter: recon uses 5 unique ports/hosts, SYN detection uses 5 TCP attempts, and UDP volume uses 500 packets. The class defaults are 20 ports/hosts, 100 TCP attempts and 1,000 UDP packets. The simulator uses a recon threshold of 6. Always identify which execution path you are discussing.

Window values expire on activity for their key, but there is no comprehensive idle-key eviction service. Therefore a time window does not guarantee globally bounded memory with unlimited unique endpoints. That is a scale-up limitation, not a reason to misstate the design.

ATTACKS, FEATURES & DETECTION

8. Reconnaissance and port scanning

Reconnaissance is information gathering before an attack. A scanner may ask "Which ports are open on this server?" or "Which machines expose this service?" A legitimate vulnerability scanner can generate the same shape.

The recon detector keeps recent contacts for each source IP. It measures unique destination ports, unique destination hosts, ports per host and hosts per port. It does not send the probes; it only analyzes observed connection records.

Vertical scan: one source contacts enough different ports on the same destination host. In upload mode, 5 or more distinct ports in the 10-second window qualify.

Horizontal scan: one source contacts the same destination port on 5 or more different hosts in the window.

Multi-host port scan: the source reaches at least 5 unique ports across at least 2 hosts, but no individual host has already crossed the vertical-port threshold. This captures distributed fan-out across hosts.

Example: a source touches ports 21, 22, 23, 80 and 443 on one server within 4 seconds. The detector recognizes 5 unique ports, compares that with the upload threshold of 5, and emits a `vertical_port_scan` alert. Ordinary repeated access to port 443 on one server does not satisfy that fan-out rule.

Confidence uses observed/threshold. At the threshold it starts at 0.70; it rises by up to 0.25 as the observed count reaches twice the threshold. Recon alert severity is medium initially and high at twice the threshold. These are configured scoring rules, not statistical proof.

Evidence includes unique destination ports or hosts, observed time span and connection attempts. The complete evidence page explains both the measurement and the comparison. Cooldown suppresses near-identical repeat alerts for 30 seconds.

The replay fix retains the alert when the threshold was crossed. A final recon snapshot can add evidence but must not replace history: a scan near the beginning of a long replay should not disappear because later events fall outside its window.

False-positive caveat: an authorized scanner is behaviourally a scanner. The current upload recon detector does not have the same exact-endpoint policy suppression used by C2 and exfiltration. An analyst must assess authorization. Say "reconnaissance-like fan-out observed," not "this source is definitely malicious."

ATTACKS, FEATURES & DETECTION

9. SYN flooding and distributed-source flooding

A SYN flood tries to overwhelm a service with TCP connection attempts that do not complete normally. Distributed attacks involve many sources. Passive observations can reveal that distribution, but do not prove address spoofing.

The detector groups TCP events by destination IP inside a 5-second window. In the upload route it requires all of these gates:

- At least 5 observed TCP connection attempts. The generic class default is 100.
- At least 80% of attempts have incomplete-state proxies S0 or REJ.
- At least 80% of attempts concentrate on the most common destination port.

S0 and REJ are flow-level proxies, not packet-by-packet confirmation of a SYN attack. A service rejection or outage can also create them. In connection normalization, an explicit conn_state is preferred. A SYN-only history with no responder bytes can infer S0; generic data history is not automatically treated as failure.

The port-concentration gate is important. Five failed attempts on five different ports look more like reconnaissance than a concentrated service flood. The upload conflict-resolution step also removes a SYN finding that overlaps a recon finding from the same source.

For the distributed subtype, the current implementation additionally requires at least 3 distinct sources and normalized source-IP entropy of at least 0.85. Entropy measures how spread the source counts are; normalization divides by $\log_2(\text{number of distinct sources})$. Equal contribution from several sources yields a value close to 1.

Measured connection rate is attempt count divided by observed span, with a small minimum denominator. Supplied syn_rate_per_sec or new_src_ips_per_sec can also appear in rate evidence. The actual decision gate remains window count, incomplete ratio and concentration; do not claim the displayed rate is an independent trained classifier.

At threshold with every connection incomplete, confidence is 0.85. Additional attempts can increase it toward 0.99. Alert severity becomes critical at twice the configured attempt threshold.

Evaluator answer: "We first detect concentrated, incomplete TCP pressure. We then describe source distribution using source count and entropy. We deliberately say distributed-source SYN flood because flow data cannot establish spoofing."

ATTACKS, FEATURES & DETECTION

10. UDP flooding and amplification

UDP flooding is high UDP traffic volume toward a target. Reflection/amplification is a more specific hypothesis: a small request corresponds to a much larger response, potentially using an intermediary service to burden another endpoint.

Generic UDP volume path: in the 5-second destination-keyed window, the detector sums outbound packet counts. It uses at least one packet per flow when packet counts are missing or zero. Upload mode uses a threshold of 500; the generic default is 1,000. The alert becomes critical at twice the threshold. This is an approximate flow-metadata volume detector, not a verified packet-capture accounting system.

Amplification path: all of these conditions must hold in that window:

- At least 4 UDP flows.
- Total responder bytes of at least 10,000.
- Responder bytes / max(originator bytes, 1) of at least 10.

Example: four UDP exchanges contain 1,000 total request-side bytes and 30,000 total response-side bytes. The ratio is 30. That crosses both the response-volume floor and ratio threshold, so amplification-shaped traffic is flagged.

The evidence card shows flow count, response/request byte ratio, UDP response bytes and source diversity. Confidence starts around 0.68 and increases with amplification strength and additional flows. The reflection/amplification alert severity is high.

Important limitation: Zeek originator/responder orientation must be understood. This implementation groups records by recorded destination and compares their counters; it does not reconstruct every reflector-victim relationship, prove source spoofing, or perform full service-aware packet attribution. A large legitimate UDP reply can resemble amplification.

The generic volume and amplification tests are separate checks. In another dataset both may emit. The current eight-behaviour fixture produces the intended amplification finding; that does not imply all future datasets always give exactly one DDoS alert.

Evaluator answer: "We identify an amplification pattern using repeated UDP exchanges and strong response-to-request byte asymmetry. We expose the evidence and uncertainty. We do not claim that byte ratios alone prove a reflected attack."

ATTACKS, FEATURES & DETECTION

11. Botnet C2 beaconing

Command and control, or C2, is communication between a compromised endpoint and infrastructure that instructs it. A beacon is a repeated callback, often at a fairly regular interval. But health checks, telemetry agents and scheduled jobs can also be periodic.

Drastha groups connections by source IP, destination IP, destination port and protocol. The window is 300 seconds. It requires at least 6 connections and applies several gates together:

- Mean inter-arrival interval between 2 and 120 seconds.
- Interval coefficient of variation, CV, at most 0.15.
- Transfer-size CV at most 0.20.
- Observation span at least 30 seconds.
- Mean total transfer size at most 2,048 bytes per connection.
- At least 80% not marked S0 or REJ.

CV means standard deviation divided by mean. For intervals 10, 10, 10, 10, 10 seconds, the variation is zero, so CV is zero. For highly uneven intervals, CV increases. Drastha also calculates sizes from originator bytes plus responder bytes, so repeated small exchanges look different from ordinary large downloads.

Example: six 1,000-byte exchanges, each 10 seconds apart, span 50 seconds. They satisfy repetition, timing, size and duration gates. They remain a C2-like hypothesis unless trusted context explains them.

DNS traffic on port 53 or service dns/domain is excluded from this detector. It belongs in DNS analysis. Exact trusted endpoint policy entries are checked before windowing. A rule can match source, destination, destination port, protocol and service. An uploaded label saying "health check" does not itself authorize suppression.

Confidence combines timing regularity and size regularity, starting at 0.68. Optional matching encrypted metadata can slightly adjust confidence when that context index is supplied. The simulator/CLI can supply such an index; the current upload constructor does not. A fingerprint is never a standalone C2 trigger.

The completed ratio is only a proxy: the code treats states other than S0/REJ as not incomplete, including an unknown state. Jittered beacons, different destinations or large payloads may evade these rules. Legitimate unlisted monitoring can still produce false positives. These are reasons for calibration and review, not claims of universal detection.

ATTACKS, FEATURES & DETECTION

12. DGA detection: ML and lexical fallback

A Domain Generation Algorithm creates many candidate names that malware might use to find a controller. Instead of contacting one fixed domain, software generates possible rendezvous domains. Some generated names look random, but dictionary-based DGAs may look readable.

Drastha has two DGA paths. First, if the saved DNS model exists, it estimates a DGA-like score for the approximate base domain. A score of at least 0.50 emits `dga_like_domain`, subject to cooldown and any configured domain allowlist.

The base-domain helper currently uses the final two labels, not a full Public Suffix List. For a name ending in `co.uk` this can select the wrong root. This limitation affects both DGA model input and tunnelling grouping.

The second path is a lexical fallback when no model score crosses the threshold. A candidate base-domain label must meet all of these:

- Length at least 10 characters.
- Shannon entropy at least 3.0.
- Digit ratio at least 0.15.
- Vowel ratio at most 0.25.

At least 3 distinct suspicious roots from the same source must appear within 60 seconds before the fallback alerts. This is stronger than labelling one unusual name malicious. It supports the mixed fixture even when the small model does not recognize a particular synthetic pattern.

Entropy measures character distribution, not maliciousness. A name with many repeated characters tends to have low entropy; a more varied label tends to have higher entropy. Digit ratio is digits/characters; vowel ratio is vowels/characters. CDN and tracking identifiers can share these characteristics.

Model-based evidence includes `model_probability`, the evaluated domain and entropy for context. Lexical fallback evidence instead includes distinct suspicious domains, average entropy, digit ratio and vowel ratio. Inspect the evidence to tell which route fired. The simulator's generic DGA method caption can say "Character n-gram ML model" even when the fallback is involved; do not rely only on that caption.

Evaluator answer: "DGA classification is a hybrid. A supervised character model produces a score, and a multi-domain lexical fallback covers additional generated-looking patterns. Neither is proof of malware, and both require realistic benign validation."

ATTACKS, FEATURES & DETECTION

13. DNS tunnelling

DNS tunnelling uses DNS messages to carry communication that is not ordinary name resolution. Encoded chunks can appear in changing subdomains under an attacker-controlled domain. This can support C2 or data transfer. Drastha does not decode the data or prove what was transferred.

The detector groups queries by source IP and approximate base domain over 60 seconds. It inspects the leftmost query label, counts queries and distinct labels, and calculates mean label length and Shannon entropy.

Normal tunnelling path requires:

- At least 20 queries to the base domain.
- At least 15 distinct leftmost labels.
- Average label length at least 18 OR average entropy at least 3.5.

A stronger encoded-TXT path permits fewer queries, but needs all of these:

- At least 4 queries.
- TXT share of at least 75%.
- Average label length at least 24.
- Average entropy at least 3.5.

Example: many long changing labels under transfer.example.test are more suspicious than repeatedly resolving one stable host. Four long, varied TXT query labels can satisfy the strong path even though they do not meet the normal 20-query threshold.

This explains a possible evidence-page confusion: an alert can show a query count below 20 and still be valid because the alternate TXT path fired. The evidence template displays some normal-path comparison text. Explain which complete rule branch was satisfied instead of saying every displayed comparison is always required.

False positives include CDN-generated names, endpoint security telemetry and legitimate DNS-based services. The detector supports base-domain allowlists when explicitly configured, but the current upload path does not populate a domain allowlist from the C2/exfiltration context-policy file.

Visibility matters. If DNS is encrypted inside HTTPS and no monitoring-side DNS metadata is supplied, this detector cannot read its query name. It does not decrypt DNS-over-HTTPS.

Current ingestion caveat: the dedicated DNS normalizer accepts qtype/rcode fields, but numeric codes are not comprehensively translated into names. The upload helper can use UNKNOWN unless a named field or feature record_type is supplied. For the TXT-specific branch, named qtype_name: "TXT" is the reliable supported representation today.

ATTACKS, FEATURES & DETECTION

14. Encrypted-session anomaly and exfiltration

Encrypted-session anomaly: TLS protects payload content, but metadata can still be informative. JA3/JA3S and JA4 are fingerprint identifiers derived from handshake characteristics. They are not malware verdicts, and different applications can share a fingerprint.

The detector groups source, destination and client fingerprint in a 60-second window. It requires at least 4 sessions. It averages supplied prevalence and anomaly features, then requires fingerprint prevalence ≤ 0.01 , packet_size_sequence_anomaly ≥ 0.75 and timing_sequence_anomaly ≥ 0.75 . All three conditions are required. Cooldown is 120 seconds.

Crucial implementation fact: this prototype consumes those prevalence/sequence anomaly scores from features/ml_evidence/evidence. It does not currently train a sequence model or calculate these scores directly from raw packet sequences. They must come from a trusted monitoring-side extractor; the synthetic fixture supplies them. Missing prevalence defaults to common and missing anomaly scores default to zero, so absence does not automatically create an alert.

Exfiltration: unauthorized information transfer may create unusually large originator-side bytes compared with responder-side bytes. The detector groups source/destination over 300 seconds and maintains up to 50 prior per-flow outbound-byte values for each source. It compares the current average with the prior median.

There are three exfiltration paths:

- Single flow: at least 10,000,000 outbound bytes and outbound/inbound ratio ≥ 20 .
- Baseline path: at least 3 flows, total outbound $\geq 1,000,000$, ratio ≥ 8 , and mean outbound bytes per flow ≥ 4 times a positive prior source median.
- Baseline-free extreme path: at least 3 flows, total outbound $\geq 10,000,000$ and ratio ≥ 20 .

The denominator is $\max(\text{inbound bytes}, 1)$. Exact approved transfer endpoints are checked first. The prior 17.8 MB backup-shaped example can be suppressed only by trusted configured context, not by an uploaded benign label.

Baseline updates include observed values, so drift or malicious traffic can contaminate the baseline. Destination age is explanatory context, not an independent trigger. Without internal/external network classification, "outbound" means the originator direction, not necessarily crossing the enterprise perimeter. Say "possible exfiltration from flow-volume asymmetry," not "we know which documents were stolen."

ATTACKS, FEATURES & DETECTION

15. The ML model: exactly how it is trained

The implemented trained model is DNSNgramModel: a custom character 3-gram multinomial Naive Bayes binary classifier. It is supervised learning because training examples have known labels: 0 for benign and 1 for malicious/DGA-like. It is not deep learning, reinforcement learning, clustering or an LLM.

Data source: examples/dns_training_demo.csv has 32 rows. Training uses 20 examples: 10 benign names and 10 synthetic malicious-looking names. Testing uses 12 examples: 6 benign and 6 synthetic malicious-looking names. Benign examples include recognizable service domains; the fixture is still a tiny demonstration dataset, not a licensed production threat benchmark.

Rows have domain, label, family and split. Examples include github.com labelled 0 in train, xq7v9k2m4p8z1c6n.biz labelled 1 in train, and v8m2q6z9x4c7n1k5.biz labelled 1 in test. These are inspected as strings; the trainer does not visit or resolve them.

Training steps:

- Normalize domain text to lowercase and remove a trailing dot.
- Add boundary markers ^ and \$. For abc.com, grams include ^ab, abc, bc., c.c, .co, com and om\$.
- Count each gram separately in benign and malicious training classes.
- Record class document counts, total gram counts and vocabulary size.
- Save counts and parameters to output/models/dns_dga_demo.json.

Prediction adds log prior probability and the log probability of each observed gram for each class. Laplace smoothing uses $\alpha = 1$ to avoid zero probability for unseen grams. The two scores are converted into a normalized malicious-class score. Log-space computation avoids multiplying many tiny numbers directly.

The model learns gram frequencies, not eight threat families. Domain length, entropy, vowel ratio and digit ratio are separate engineered lexical features used by fallback rules or evidence; they are not numeric input columns in this Naive Bayes model.

Library answer: Python's csv, json, math and collections.Counter implement training and inference. The repository's analytics core has no mandatory third-party ML dependency. This keeps it small, offline and inspectable, but does not make it a state-of-the-art detector.

ATTACKS, FEATURES & DETECTION

16. Training validation and honest accuracy claims

The dataset reader checks for identical domains appearing across train/test and for malicious family identifiers shared between the splits. Training consumes only rows marked train. Testing consumes only rows marked test. These checks reduce obvious leakage.

Fresh re-evaluation of the 32-row demonstration data gave 6 TP, 0 FP, 6 TN and 0 FN on its 12 test examples. The stored model card also reports accuracy, precision, recall and F1 of 1.0. This proves that this small fixture and training pipeline behave as intended, not that the classifier generalizes to real Internet traffic.

Family labels in this fixture are synthetic grouping labels. Different labels do not make the examples diverse real malware families. The malicious strings share obvious shapes; legitimate domains are much more varied in production. There is also a possible mismatch between full domains in offline testing and approximate base domains passed by the detector.

Implemented measures that improve reliability:

- Separate training and test rows with leakage checks.
- Laplace smoothing for unseen character grams.
- Log-space score calculation for numerical stability.
- A multi-domain lexical fallback, instead of relying entirely on a tiny model.
- Benign-pattern tests and exact operational policies for the appropriate detectors.

Not implemented as a validated accuracy-improvement system: cross-validation over large datasets, automated hyperparameter search, calibrated probabilities, continuous retraining, drift detection, or model updates from analyst feedback.

Future production approach: collect versioned, licensed benign and DGA data; hold out real malware families and later time periods; deduplicate related names; include CDNs, tracking, telemetry and organizational domains as difficult negatives; tune on a validation split; freeze a final test set; measure per-family recall and operational false-positive rates; calibrate scores on deployment-like data.

Do not tune repeatedly on the final evaluation set and then call it unseen validation. Do not insert evaluation labels into feature extraction. Fields named `ml_evidence` can contain numeric metadata, but that name does not prove those values were generated by an ML model.

Evaluator answer: "Our current trained component is deliberately inspectable. The tiny synthetic test is a smoke test. Real deployment accuracy requires a much larger and independent validation program."

ATTACKS, FEATURES & DETECTION

17. How false alerts are reduced

A false positive is benign behaviour flagged as suspicious. Drastha reduces false positives through threat-specific evidence, operational context and review. It does not guarantee their elimination.

Problem	Implemented safeguard	Remaining risk
Normal HTTPS called C2	Small transfers, long observation span, stable timing/size, not-incomplete ratio	Legitimate periodic agents can still match
Health checks called C2	Exact trusted periodic endpoint rules	A compromised approved service could be hidden
Backups called exfiltration	Exact approved bulk-transfer endpoint rules and baseline comparison	Policy must stay current and narrow
Port scan called SYN flood	Destination-port concentration and upload overlap resolution	Mixed attacks can be more complex
One rare TLS fingerprint called malware	Repeated sessions plus both timing and size anomaly gates	Supplied feature quality limits reliability
One odd domain called DGA via fallback	Three distinct suspicious roots in a window	Model path can still alert on a single name
Repeated identical notifications	Cooldowns and upload finding merging	Merging is not the same as improving accuracy

The context policy is monitoring-side configuration in `config/context_policy.json`, optionally selected by `DRASTHA_CONTEXT_POLICY`. Endpoint matches include source IP, destination IP, port, and configured protocol/service. It is not a live reputation lookup. The current upload path does not fetch reputation from the Internet or observed hosts.

This distinction matters: `evaluation_label: benign` is an answer key for scoring; it must not suppress a detector. Similarly, a raw text claim saying "approved backup" is not automatically trusted. Operational approval comes from a separate configured policy.

Policies can create false negatives. If a trusted endpoint is compromised, broad or stale approvals may hide suspicious activity. Use narrow rules, expiry/review procedures and provenance in production. Those operational controls are recommendations, not all automated today.

Analyst feedback is stored as `confirmed_malicious`, `benign` or `needs_review`. It supports investigation and future validation work. It does not currently retrain the model or automatically rewrite allowlists. Changing an incident to `false_positive` also does not retroactively recompute a stored evaluation history, because no such durable run-history subsystem exists.

Best evaluator answer: "We reduce false alarms by requiring several independent behavioural conditions and using narrowly configured operational context. We keep the evidence and alternatives visible so the analyst can confirm the interpretation."

18. From detector output to a standardized alert

A detector can receive many normal events and emit nothing. When a rule crosses its boundary, it builds an Alert object. Detection is not performed by React and not by database SQL queries.

An alert contains `alert_id`, `detector_id`, `detector_version`, `threat_type`, `subtype`, `confidence`, `severity`, `window_start`, `window_end`, `src_ip`, `dst_ip`, `flow_ids`, `evidence` and `limitations`.

`to_dict()` converts this object into a JSON-compatible record and adds interoperability fields: `schema_version`, `timestamp`, `flow_identifier`, `threat_class` and `supporting_evidence`. `timestamp` is the alert window end. `flow_identifier` is one representative flow ID; `flow_ids` can contain several contributing records.

Evidence is structured as four fields:

- `name`: the measured feature, such as `interval_variation`.
- `observed`: its actual value, such as 0.03.
- `comparison`: the rule boundary or "context".
- `explanation`: why the feature matters.

An alert ID is a shortened SHA-256 digest of detector-specific identifying strings such as source, destination, subtype and time span. It helps deterministic identity and replay upserts. It is not a digital signature of the original capture and does not establish forensic chain of custody.

Upload processing merges findings with the same family, subtype, source and destination. It keeps maximum confidence, earliest start, latest end and the union of flow IDs. Evidence comes primarily from the strongest representative alert, with a `related_findings_merged` count added. The code does not recompute every evidence statistic over the merged interval.

This matters in a demo: a merged finding's time span and flow set may be wider than the precise evidence window that first crossed a threshold. Do not claim all displayed evidence has been recalculated over every merged flow.

Alert confidence is generally a heuristic formula. For a model-based DGA alert it is the model score, but even that score is not calibrated on a real deployment population. A 0.90 score should be described as "strong detector support under the current scoring design," not "a guaranteed 90% chance of malware."

The upload merge key has no separate episode-time boundary. Long independent episodes with the same endpoint/subtype can be merged within one upload. This is a prototype simplification to understand when discussing incident granularity.

INTELLIGENCE, APIS & DASHBOARD

19. Incident correlation, severity and risk scoring

An alert is one detector finding. An incident is an analyst investigation group containing related alerts. Eight alerts can become fewer than eight incidents if they share the same source and fall inside the correlation relationship.

IncidentStore groups by source IP with a 900-second correlation window. It checks temporal proximity to an existing incident and avoids adding the same alert ID twice. This is a simple source-centric correlation model, not full graph-based attribution across NAT, victims, reflectors or distributed botnets.

Current incident risk formula:

$\text{Risk} = \min(100, \text{distinct threat weights} + \text{cross-detector bonus} + \text{confidence component})$.

Threat weights: reconnaissance 15; denial_of_service 25; dns_threat 25; command_and_control 35; data_exfiltration 40. Any family absent from the map gets 10. Encrypted_session_threat currently falls back to that 10-point weight.

Cross-detector bonus = $8 \times \max(\text{number of distinct detector IDs} - 1, 0)$. Confidence component = $\text{round}(\text{mean alert confidence} \times 20)$. Several alerts from the same detector do not generate extra distinct-detector bonuses. DGA and tunnelling share dns.analytics, so they count as one detector ID.

Incident priority bands: critical ≥ 80 ; high ≥ 60 ; medium ≥ 35 ; otherwise low. Individual alert severity is set separately inside each detector. Therefore a high-severity alert can belong to a medium- or low-priority incident under the current formula.

Worked example: suppose one source has a C2 alert at 0.85 confidence and an exfiltration alert at 0.92. Threat weights = $35 + 40 = 75$. Two detectors add 8. Mean confidence is 0.885; the confidence component is 18. Total is 101, capped at 100: a critical incident.

Fresh mixed-replay examples show this distinction: the distributed SYN alert has confidence 0.85 and alert severity high, but its single-family incident score is $25 + 17 = 42$, which is medium. The encrypted anomaly alert is high, yet its incident score is $10 + 18 = 28$, which is low. This is the current policy, not an ML conclusion.

Do not call these "independent probabilities." The UI's independent-check count means different detector IDs; their evidence may be correlated. Risk is a prioritization aid and still needs asset criticality, deployment tuning and operational review.

INTELLIGENCE, APIS & DASHBOARD

20. APIs: the bridge between browser and backend

An API is an agreed way for one component to ask another component for data or an operation. The browser sends HTTP requests; FastAPI validates the request and calls Python services. The browser never queries SQLite directly.

Endpoint	Purpose
POST /api/replays/analyse	Validate/analyze uploaded filename and content; persist alerts/incidents; return report
GET /api/incidents	Read the investigation queue, optionally filtered by status/severity
GET /api/incidents/{id}	Load an incident, its alerts and analyst feedback
GET /api/metrics	Database-wide counts and average risk
PATCH /api/incidents/{id}/status	Change investigation workflow state
POST /api/incidents/{id}/feedback	Store an analyst decision and notes
GET /api/incidents/{id}/export	Return structured JSON evidence export
GET /api/stream/simulated	Stream live demonstration messages over SSE
GET /api/health	Report application mode, storage kind and last demo-run summary
GET /api/replays/sample	Download the configured sample replay
POST /api/demo/run or /api/demo/load	Run the demo story or load bundled demo records

HTTP 200 means a normal successful response. Feedback creation uses 201. Unknown incident IDs use 404. Validation problems generally use 422. A demo unable to run can return 503.

Pydantic models validate basic request structure and limits. The replay code performs deeper telemetry normalization. Uvicorn is the application server that hosts FastAPI; FastAPI is the framework defining routes and request handling.

SSE means Server-Sent Events. The browser uses EventSource and the backend emits started, traffic, alert and complete messages through one HTTP response. This monitoring-side stream is distinct from the one-way production ingest boundary. It is not a command channel back to observed hosts.

The /api/health response's top-level healthy string is a simple service response, not a comprehensive database/telemetry health assessment. The uploaded replay's quality.status is computed separately. Never use one as proof of the other.

The current API is an offline/demo-oriented implementation. CORS configuration is not authentication. Role-based access, authentication, production TLS termination, rate controls and deployment isolation should be added before exposure beyond a trusted environment.

INTELLIGENCE, APIS & DASHBOARD

21. Database design and configuration

The default API repository is SQLite at output/draस्था.db relative to the process working directory. Demo-serve defaults to a different configured file, output/draस्था-demo.db. DRAस्था_DB overrides the selection; AEGISFLOW_DB is a legacy fallback. Do not assume every running instance uses the same database file.

If the value begins with postgres:// or postgresql://, repository_from_url selects the PostgreSQL adapter instead. The adapter uses psycopg and the same logical schema. It is available infrastructure, not proof that a PostgreSQL deployment has been load-tested.

Table	Purpose and key data
incidents	incident_id primary key, source, times, risk, severity, confidence, status, JSON payload
alerts	alert_id primary key, incident_id association, family, severity, start, complete JSON payload
analyst_feedback	feedback_id, incident_id, disposition, analyst, timestamp, notes
runtime_state	state_key primary key, JSON state payload and update time

JSON payloads are stored as text in this shared schema. Important scalar columns are separately indexed for queue retrieval. The SQLite connection commits successful operations and rolls back on exceptions. SQL parameters are passed as bound values. An alerts incident_id column relates the records logically; the current schema does not declare an SQL foreign-key constraint for that association.

Upserts use stable primary keys. Replaying the same deterministic alert/incident IDs updates rows rather than adding another copy. Re-importing an incident preserves the stored workflow status. This is idempotence by ID, not a global content-level duplicate removal algorithm.

The upload path persists alerts and incidents, not a raw upload archive, full normalized event history, permanent quarantine table or evaluation-run history. Quality and evaluation details are returned in the response. Existing JSONL files remain wherever the user placed them.

Selected CLI exfiltration runs can persist runtime_state. That exported state includes baseline history, destination times, cooldowns and window events, including their raw dictionaries. Therefore "raw events are never persisted anywhere" would be too broad: normal uploads do not archive them, but optional detector-state persistence can retain them.

No automatic retention schedule, encrypted-at-rest database setup, immutable evidence archive or complete chain-of-custody subsystem is established by these tables. These are production requirements to plan separately.

22. How the result reaches the dashboard

The frontend is React written in TypeScript. During development Vite serves it, commonly on port 5173. The build command runs TypeScript project checking and Vite bundling, producing static assets under web/dist. FastAPI can serve those built files using DRASTHA_WEB or its default web/dist path.

This is what "compile" means here: frontend TypeScript/React source becomes browser-deliverable JavaScript/CSS/HTML. Python detector decisions are calculated at runtime; the frontend build does not classify attacks.

Upload sequence:

- User selects the actual traffic file, not its manifest.
- React reads text and sends filename/content to the analyse API.
- Python normalizes, checks quality, runs detectors and stores intelligence.
- The API returns a JSON report with alerts, incidents, quality, timings, evaluation and stages.
- React stores the report in component state and renders the result cards.
- The frontend refreshes the incident queue and database-wide metrics through separate GET requests.

The upload result and the investigation queue have different scopes. Upload result counts refer to this request. Queue metrics can include previously stored incidents. A successful upload with eight findings does not mean the entire database contains exactly eight findings.

The stage animation is a presentation of backend-reported steps. For upload it is displayed after the API response; it is not a live progress feed from inside the upload parser. The upload reads the complete file and returns at completion. The simulated SSE route is the path that publishes incrementally as each paced event is processed.

Analysis time is measured server-side for the upload operation, including validation, detection, correlation and persistence. It excludes file selection, browser transfer and rendering. The read/check stage durations are currently placeholders at 0.0 rather than separate measured timings.

The simulator preloads and sorts demo data, then processes it event by event. Its observation-based latency still depends on detector windows. A fast CPU cannot identify a six-beacon pattern before the necessary observations exist. End-to-end bounded-latency production guarantees would require queueing, resource and backpressure measurements beyond the demonstration.

INTELLIGENCE, APIS & DASHBOARD

23. Dashboard glossary: what each term really means

Term on screen	Correct interpretation
Accepted records	Input records that produced supported events, not a count of benign records
Rejected / quarantine	Invalid record accounting, not proof of malicious traffic
Data quality	Tracked input validity/order/duplicate state, not network safety
Findings	Alerts after the current upload's conflict resolution and merging
Incidents	Correlated investigation groups, possibly containing multiple alerts
Confidence	Detector support score; generally not a calibrated probability
Severity on a finding	Detector-specific urgency category
Risk / priority	Incident policy score and its severity band
DNS records / TLS records	Routed metadata event counts; not separate original files necessarily
Policy-approved records	Sum of detector suppression evaluations; not necessarily unique input records
TP / FP / FN / TN	Behaviour/endpoint evaluation outcomes when answer-key metadata exists
Analysis time	Server-side operation duration, not total wall-clock user wait
Active	Database incidents marked open or investigating
Critical	Database incidents with critical priority; current count is not restricted to active status
Reviewed	Number of feedback records, not necessarily distinct reviewed incidents
Independent checks	Distinct detector IDs, not mathematically independent statistical tests
No configured threat pattern found	No current detector threshold crossed; not a guarantee of safety

The live section has additional terms. Records analysed is progress through the simulation. Labelled alerts is emitted findings so far. Current risk follows incident updates during the stream and the final top incident at completion; it is not a universal network-health score. Latest observation is metadata from the current event.

The row's IP address is the source field associated with that incident. In a distributed DDoS alert, the emitted src_ip can be the source of the threshold-crossing event, while the evidence contains several sources. Do not describe that one displayed IP as the only attacker.

Dashboard counts can differ without being contradictory. The mixed fixture has 452 accepted originals: 342 plain connection schema records, 79 DNS and 31 TLS. TLS also enters the upload connection-event path, producing 373 connection events plus 79 DNS events and 31 TLS metadata events. These event counts overlap; do not sum them and call it 483 original records.

24. How to read the full evidence page

Clicking Review full evidence uses an incident ID, normally the highest-risk incident from the upload. React requests GET /api/incidents/{id}. The repository returns the incident plus its alert payloads and feedback. The page does not rerun a new classifier at this moment.

Incident header: shows source, incident ID, incident risk, priority and mean detector confidence. Status is workflow state: open, investigating, resolved or false_positive. A status change is saved through PATCH; it is not a detector prediction.

What happened: a timeline of stored alerts, ordered by their window start. Read it as observed suspicious activity, not proven causal steps. Correlation by source and time does not demonstrate that one event caused another.

Why Drastha flagged it: the full evidence list for every related alert. For each item, read the measurement, comparison and explanation together. A value marked context supports interpretation but may not be a required rule gate. The upload summary card shows only the first few evidence items; the drawer shows more.

Possible alternative: a detector limitation, such as authorized scanning, legitimate monitoring or backups. The UI currently displays the first limitation in several places. The complete alert JSON/export can contain additional limitations.

Why it is prioritized: the score breakdown from incident correlation, not a second detection model. The three contributions are distinct-threat weight total, cross-detector bonus and confidence component. Sum them and apply the 100-point cap.

Record the decision: an analyst records confirmed malicious, needs review or benign, with a name and notes. This is human review evidence. It is stored but does not automatically teach the model or modify operational policies.

Export evidence: downloads JSON containing format, export timestamp and the incident with associated alerts/feedback. The export is useful for review, but is not automatically a signed, immutable forensic record.

Good demo narration: "This is the observed feature; this is the configured threshold or supporting context; this is why the pattern is suspicious; these are the alternatives; and this is how the incident priority was calculated. The analyst makes the final operational judgment."

One caution: exfiltration and DNS tunnelling have alternate rule branches, while some displayed comparisons come from a shared template. If one displayed threshold seems unmet, inspect the branch that actually triggered instead of inventing a stronger claim.

EVALUATION & PRESENTATION

25. Evaluation: records are not behaviours

The mixed v3 file contains 452 records: 91 attack-labelled records and 361 benign-labelled records. Stateful detectors combine multiple observations, so they should not be scored as if every record independently predicts a different attack.

The evaluation layer runs after detector inference. It reads `evaluation_label`, `evaluation_threat_class` and `uid`. Those fields form the answer key; they do not choose detector thresholds or approve traffic. The current grouping logic does not use `evaluation_id` to split independent episodes, despite preserving that field in records.

Grouping is target-centric for flood behaviours, source-centric for recon, source/destination-centric for other attacks, and endpoint/port/service-centric for benign traffic. An alert overlaps a unit when its flow IDs intersect that unit. Correct attack subtype overlap is a TP. A missed expected attack is FN. An alert overlapping a benign unit is FP. An unalerted benign unit is TN. Unmatched alerts also contribute FP.

The current fixture forms 94 units, not 452 independent classification outcomes. Fresh verification returned 8 TP, 0 FP, 0 FN and 86 TN, with eight findings and eight incidents. All 452 were accepted; quality was healthy and duplicate/out-of-order/rejected counts were zero.

Precision = $TP / (TP + FP)$: how many flagged units were correct. Recall = $TP / (TP + FN)$: how many expected attack units were found. $F1 = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$. $FPR = FP / (FP + TN)$: how often benign units were flagged.

For 8 TP, 0 FP and 0 FN, precision, recall and F1 are 100%. With 86 TN and no FP, FPR is 0%. The valid statement is "100% behaviour-level precision and recall on this controlled mixed fixture," not "100% accuracy against cyberattacks."

Two source identities occur in both benign and attack portions. The detector is not reset using labels. The result therefore checks this particular stateful replay, but independent scenario evaluation should use explicit external boundaries and broader identities. Incorrect extra subtypes on an attack unit are not equivalent to a complete multiclass confusion matrix; review classification mismatches and extra findings separately.

The v3 ordering fix preserves all record objects. Original telemetry had one 320-second backwards step and ten behind-maximum records. Correcting the input order fixed quality without changing the eight detections.

EVALUATION & PRESENTATION

26. Throughput, tests and reproducibility

Functional tests check specific behaviour: parsing, normalization, detector boundaries, benign negatives, correlation, storage upserts, API workflows, replay quality, PCAP command construction and restart state. The previous completed full run in this project reported 115 tests passing. A passing test suite is not a network-scale security certification.

Fresh checks used while preparing this handbook reran the v3 upload-analysis function and the DNS model evaluation. They reconfirmed 452 accepted records, healthy quality, eight findings/incidents and 8/0/0/86 behaviour outcomes; the model smoke test reconfirmed 6/0/6/0.

The declared prototype throughput target is 1,000 records per second. The stored output/`drastha_evaluation_report.json` contains a historical 250-iteration run over 67 records per iteration: 16,750 records, 1,340.82 sustained events/second, median 47.564 ms and P95 60.519 ms per iteration.

That benchmark includes fixture file reading, normalization, detection, correlation, SQLite upserts and SSE serialization. It excludes live packet capture, Zeek conversion, HTTP transport and browser rendering. Do not present its rate as Mbps or end-to-end physical-link throughput.

The stored report also contains older suspected-spoofed-source subtype wording. Current source uses `distributed_source_syn_flood`. Therefore treat that benchmark as a historical artifact, not a freshly regenerated benchmark for the exact latest source. Older documentation may quote another run; always identify the artifact, scope and environment you are quoting.

Useful commands from the project root:

- `.venv\Scripts\python.exe -m unittest discover -s tests -v`
- `.venv\Scripts\python.exe -m unittest tests.test_mixed_evaluation_replay -v`
- `python -m aegisflow.validate_replay examples/drastha_mixed_evaluation_v3.jsonl`
- `drastha evaluate-demo --iterations 250 --report-output output/drastha_evaluation_report.json`
- In web: `pnpm run build`

The validation CLI uses the upload analysis with a no-op repository sink, so it checks the same route without importing incidents. It still executes detector analysis; it is not a completely separate lightweight parser-only tool.

For a presentation, use the v3 JSONL rather than the manifest or the original out-of-order JSON. Test the actual launch mode and database before the demonstration. Avoid promising timing figures from a different machine or from a different measurement scope.

EVALUATION & PRESENTATION

27. Difficult evaluator questions and safe answers

Q: Is the whole solution AI/ML? A: No. It is hybrid. DGA has a supervised character Naive Bayes model. Most other detectors use transparent statistical/behavioural rules. Exfiltration uses an adaptive median baseline; TLS consumes supplied anomaly metadata.

Q: Why not one model for everything? A: Different attacks require different evidence. DNS lexical patterns, TCP incomplete-state concentration and byte asymmetry are not interchangeable features. Separate detectors make the assumptions and explanations inspectable.

Q: Can you prove malware in TLS? A: No. We flag repeated metadata anomalies without decrypting. Rare fingerprints alone are insufficient. Sequence/prevalence scores must be produced by a trusted monitoring-side extractor.

Q: Is your 100% result real-world accuracy? A: No. It is behaviour-level performance on one controlled fixture. The 12-example DNS model test is a separate tiny smoke test. Broader independent datasets and operational calibration are required.

Q: How do you avoid health-check false alarms? A: We combine timing with size, duration and state gates, then match narrow configured operational endpoints. We do not trust an input label that merely says benign.

Q: Does "read-only" mean nothing is written anywhere? A: No. Input observations are not modified and no response is sent to observed hosts. Alerts, incident state and analyst decisions are written inside the monitoring environment.

Q: Can it stop attacks? A: It produces intelligence. Inline blocking and sending mitigation commands across the ingest boundary are outside scope.

Q: Is upload streaming? A: The detector interface is incremental, but the upload endpoint processes the complete submitted file before responding. The simulated SSE route demonstrates incremental event processing and publication. It is not yet a production unbounded stream service.

Q: What if DNS is encrypted? A: The DNS detector needs visible or separately supplied DNS metadata. It cannot inspect names hidden inside encrypted DNS by decrypting them.

Q: Does feedback retrain your model? A: No. Feedback is persisted for review and future dataset curation. Automatic retraining is not implemented.

Q: Why is a high alert in a low-priority incident? A: Alert severity and incident risk use different formulas. The current encrypted family uses a default incident weight. These policy choices need review before production.

Q: Is PostgreSQL, authentication or a data diode fully deployed? A: PostgreSQL has an adapter; authentication and physical isolation are deployment work. We should show what is actually configured rather than imply the prototype provides these automatically.

EVALUATION & PRESENTATION

28. Presentation script and final revision sheet

Opening, about 45 seconds: "Drastha is a passive threat-intelligence prototype for a one-way monitoring environment. We accept observed connection, DNS and encrypted-session metadata, validate its quality, run threat-specific analytics, and present labelled alerts with confidence and supporting evidence. We do not contact the monitored hosts or decrypt payloads."

Architecture, about 60 seconds: "The browser sends file text to a FastAPI endpoint. A parser extracts records from JSONL or JSON containers. Alias normalization produces canonical Zeek-shaped fields. Schema routing separates DNS, connection and encrypted metadata. Quality is measured in original input order. The detectors use event-time windows, then findings are deduplicated, correlated and stored. React reads the result and the incident API to display evidence."

Detection, about 90 seconds: "Recon looks for port/host fan-out. SYN flooding needs concentrated incomplete attempts; source entropy describes distribution. UDP amplification looks for large response/request ratios. C2 needs sustained regular small exchanges. DNS uses a small n-gram model, lexical fallback and query-shape analysis. Encrypted anomaly requires repeated rare fingerprints plus supplied timing/size evidence. Exfiltration compares transfer volume and direction ratios against a source baseline or extreme thresholds."

Evidence, about 45 seconds: "This card shows the actual feature, its comparison and an explanation. The incident score is a policy sum, not a probability. The analyst can inspect alternative explanations, record a decision and export JSON evidence."

Validation, about 30 seconds: "On this 452-record chronological fixture, all eight intended attack behaviours are detected with no false-positive behaviour units. The result is 8 TP, 0 FP, 0 FN and 86 TN. This is controlled scenario validation; production accuracy needs much broader testing."

Before presenting, answer these without notes:

- Why do quality and maliciousness differ?
- What is the difference between a packet, a flow, an alert and an incident?
- What is actually learned by the DGA model?
- Which TLS features are supplied rather than calculated here?
- What distinguishes a scan from a concentrated SYN flood?
- Why does an approved backup need external policy context?
- What is saved to the database, and what exists only in the upload response?
- What is the difference between confidence, severity, priority and measured precision?

If you do not know an evaluator's answer, say: "That capability is not verified in this prototype. The implemented path is X; adding Y would require Z." A precise limitation is more credible than an invented implementation claim.

EVALUATION & PRESENTATION

29. Source map for technical follow-up

All paths below are relative to the Drastha repository root, F:\Drastha\Drastha. These source files, not generic product descriptions, were used to explain the current implementation. No production source code was changed while preparing this handbook.

Topic	Source files
Browser workflow and evidence drawer	web/src/App.tsx; web/package.json
API routes and request models	src/aegisflow/api.py
Upload sequence, quality, merge rules	src/aegisflow/upload_analysis.py
Container parsing and schema routing	src/aegisflow/ingestion/replay_input.py
Aliases and connection validation	src/aegisflow/ingestion/zeek_jsonl.py
DNS and encrypted normalization	src/aegisflow/ingestion/zeek_dns.py; zeek_encrypted.py
PCAP/Zeek file conversion	src/aegisflow/ingestion/zeek_runner.py
Event-time windows	src/aegisflow/windowing.py
Recon, DDoS, C2	src/aegisflow/detectors/recon.py; ddos.py; c2.py
DNS, encrypted, exfiltration	src/aegisflow/detectors/dns.py; encrypted.py; exfiltration.py
DGA features, model and training	src/aegisflow/dns_features.py; dns_model.py; dns_training.py
Training examples and model card	examples/dns_training_demo.csv; output/DNS_MODEL_CARD.md
Saved model and model metrics	output/models/dns_dga_demo.json; output/dns_model_metrics.json
Context and approved endpoints	src/aegisflow/context_policy.py; config/context_policy.json
Alert schema and incident risk	src/aegisflow/models.py; incidents.py
SQLite/PostgreSQL repositories	src/aegisflow/api_store.py; postgres_store.py
Simulation and validation CLI	src/aegisflow/streaming_demo.py; validate_replay.py; cli.py
Evaluation grouping and metrics	src/aegisflow/evaluation_scoring.py; evaluation.py
Mixed fixture regression	examples/drastha_mixed_evaluation_v3.jsonl; tests/test_mixed_evaluation_replay.py
Ingestion regression matrix	tests/test_universal_ingestion.py; tests/fixtures/

Revision note: the application package is named `drastha`, while the Python import package remains `aegisflow`. That is a naming detail, not two separate analytics products. Exact thresholds and behaviour can change in later commits; recheck this source map before quoting numbers after an update.